

• 오라클 10일차

- DROP TABLE hr.dw_emp PURGE;
- CREATE TABLE hr.dw_emp
- TABLESPACE users
- AS
- SELECT employee_id, last_name, salary, department_id
- FROM hr.employees
- WHERE department_id = 20;
- CREATE TABLE hr.oltp_emp
- TABLESPACE users
- AS
- SELECT employee_id, last_name, salary, department_id
- FROM hr.employees;
- SELECT * FROM hr.dw_emp;
- SELECT * FROM hr.oltp_emp;
- SELECT * FROM user_objects;
- SELECT * FROM user_tables;
- SELECT * FROM user_tab_columns;
- SELECT * FROM user_constraints;
- SELECT * FROM sys.obj\$ WHERE name = 'DW_EMP';
- SELECT * FROM dba_objects WHERE object_name = 'DW_EMP';
- SELECT * FROM sys.tab\$ WHERE obj# = 20359;
- SELECT * FROM dba_tables WHERE table_name = 'DW_EMP';
- SELECT * FROM sys.col\$ WHERE obj# = 20359;
- SELECT * FROM dba_tab_columns WHERE table_name = 'DW_EMP';
- SELECT * FROM sys.con\$ WHERE owner# = 43;
- SELECT * FROM dba_constraints WHERE owner = 'HR' and table_name = 'DW_EMP';
- desc hr.oltp_emp
- -- 기존 테이블 컬럼 추가
- ALTER TABLE hr.oltp_emp ADD flag char(1);
- SELECT * FROM hr.dw_emp;
- SELECT * FROM hr.oltp_emp WHERE employee_id IN (201,202);
- UPDATE hr.oltp_emp
- SET flag = 'D'
- WHERE employee_id = 202;

- UPDATE hr.oltp_emp
- SET salary = 20000
- WHERE employee_id = 201;

- ROLLBACK;

- COMMIT;

- /* 문제 oltp_emp에 있는 사원들 중에 dw_emp에 존재하는 사원 정보를 출력
- 문제 dw_emp에 있는 사원들중에 oltp_emp에 존재하는 사원들은
- oltp_emp에 있는 급여를 기준으로 10% 인상, 테스트한 후 ROLLBACK
진행
- 문제 dw_emp에 있는 사원들중에 oltp_emp에 존재하는 사원이면 flag 값이
'd'에
- 대해서 삭제, 테스트 후 ROLLBACK
- 문제 oltp_emp테이블에 있는 데이터 중에 dw_emp 테이블에 없는
데이터들을 \
- dw_emp테이블에 입력 테스트 후 ROLLBACK */

- -- 1
- SELECT * FROM hr.oltp_emp
- WHERE employee_id IN (SELECT employee_id FROM hr.dw_emp);
-
- -- 2
- INSERT INTO hr.dw_emp(employee_id, last_name, salary, department_id)
VALUES (300,'Liam',5000,NULL);
- INSERT INTO hr.dw_emp(employee_id, last_name, salary, department_id)
VALUES (400,'James',6000,NULL);

- SELECT *
- FROM hr.dw_emp d
- WHERE EXISTS (SELECT 'a'
- FROM hr.oltp_emp
- WHERE employee_id = d.employee_id);

- UPDATE hr.oltp_emp o
- SET salary = (SELECT salary * 1.1
- FROM hr.oltp_emp
- WHERE employee_id = o.employee_id)
- WHERE EXISTS (SELECT 'a'
- FROM hr.oltp_emp
- WHERE employee_id = o.employee_id);

- ROLLBACK;

- -- 3
- SELECT o.*

- FROM hr.oltp_emp o JOIN hr.dw_emp d
- ON d.employee_id = o.employee_id
- WHERE o.flag = 'D';

- DELETE FROM hr.oltp_emp o
- WHERE o.employee_id IN (SELECT d.employee_id FROM hr.dw_emp d)
- AND o.flag = 'D';

- ROLLBACK;

- -- 4

- SELECT *
- FROM hr.oltp_emp o
- WHERE NOT EXISTS (SELECT 'a'
- FROM hr.dw_emp
- WHERE employee_id = o.employee_id);
-
- INSERT INTO hr.dw_emp(employee_id, last_name,salary,department_id)
- SELECT employee_id, last_name, salary,department_id
- FROM hr.oltp_emp o
- WHERE NOT EXISTS (SELECT 1 FROM hr.dw_emp d WHERE
- d.employee_id = o.employee_id);

- ROLLBACK;

- MERGE(9i)
 - INSERT, UPDATE, DELETE 문을 한꺼번에 수행하는 문

 - SELECT *
 - FROM hr.dw_emp d
 - WHERE EXISTS (SELECT 'a'
 - FROM hr.oltp_emp
 - WHERE employee_id = d.employee_id);

 - SELECT * FROM hr.dw_emp;
 - SELECT * FROM hr.oltp_emp;

 - SELECT d.*, o.*
 - FROM hr.dw_emp d, hr.oltp_emp o
 - WHERE d.employee_id = o.employee_id;

 - -- WHEN MATCHED , WHEN NOT MATCHED 중 하나는 꼭 실행해야지 된다.
 - MERGE INTO hr.dw_emp d
 - USING hr.oltp_emp o
 - ON (d.employee_id = o.employee_id)
 - WHEN MATCHED THEN

- UPDATE SET d.salary = o.salary * 1.1
- DELETE WHERE o.flag = 'D'
- WHEN NOT MATCHED THEN
- INSERT (d.employee_id, d.last_name, d.salary, d.department_id)
- values (o.employee_id, o.last_name, o.salary, o.department_id);
- ROLLBACK;
- DROP TABLE hr.emp PURGE;
- CREATE TABLE hr.emp(id number, name varchar2(30), day date) TABLESPACE users;
- SELECT * FROM user_tables WHERE table_name = 'EMP';
- SELECT * FROM user_tab_columns WHERE table_name = 'EMP';
- -- 컬럼 추가
- ALTER TABLE hr.emp ADD job_id varchar2(30);
- -- 컬럼 타입, 크기 수정
- ALTER TABLE hr.emp MODIFY job_id varchar2(40);
- -- 컬럼 삭제
- ALTER TABLE hr.emp DROP COLUMN job_id;
- -- SET UNUSED
- -- 사용하지 않은 컬럼으로 표시
- SELECT * FROM hr.emp;
- ALTER TABLE hr.emp SET UNUSED COLUMN job_id;
- -- 주의 SET UNUSED를 지정한 컬럼은 되돌릴 수 없음
- -- 복구작업은 백업리커버리를 이용
- desc hr.emp;
- SELECT * FROM user_unused_col_tabs;
- -- 이로인해 작업하는 내용에 대한 일지를 작성해야한다.
- -- SET UNUSED 를 수행한 컬럼 삭제
- ALTER TABLE hr.emp DROP UNUsED COLUMNS;
- SELECT * FROM hr.emp;
- INSERT INTO hr.emp (id,name,day)values (1,'Liam',sysdate);
- INSERT INTO hr.emp (id,name,day)values (1,'James',sysdate);
- INSERT INTO hr.emp (id,name,day)values (NULL,'Scott',sysdate);
- SELECT * FROM hr.emp;

- ROLLBACK;
- -- 제약조건
 - -- 데이터에 대한 규칙을 적용
 - -- 데이터의 품질을 향상시키기 위해
 - -- **Primary key**
 - -- 테이블의 대표키
 - -- 테이블당 하나만 존재 가능
 - -- **Unique** (유일한 값), Null값은 허용이 안된다.
 - -- 자동으로 **unique index** 생성
 - -- 제약조건 추가
 - ALTER TABLE hr.emp ADD CONSTRAINT emp_id_pk PRIMARY KEY (id);
 - -- &
 - ALTER TABLE hr.emp ADD PRIMARY KEY(id);
 - -- 제약조건 삭제
 - ALTER TABLE hr.emp DROP CONSTRAINT emp_id_pk;
 - --&
 - ALTER TABLE hr.emp DROP CONSTRAINT SYS_C007011;
 - --&
 - ALTER TABLE hr.emp DROP PRIMARY KEY;
 - -- 제약조건 정보
 - SELECT * FROM user_constraints WHERE table_name = 'EMP';
 - SELECT * FROM user_cons_columns WHERE table_name = 'EMP';
 - -- 인덱스 정보
 - SELECT * FROM user_indexes WHERE table_name = 'EMP';
 - SELECT * FROM user_ind_columns WHERE table_name = 'EMP';
 - INSERT INTO hr.emp (id,name,day)values (1,'Liam',sysdate);
 - INSERT INTO hr.emp (id,name,day)values (1,'James',sysdate); -- 오류발생, Primary key 제약조건 위반
 - INSERT INTO hr.emp (id,name,day)values (NULL,'Scott',sysdate);
 - SELECT * FROM hr.emp;
 - ROLLBACK;
 - CREATE TABLE hr.dept(dept_id number, dept_name varchar2(30))
TABLESPACE users;
 - ALTER TABLE hr.dept ADD CONSTRAINT dept_pk PRIMARY KEY(dept_id);
 - -- 제약조건 정보
 - SELECT * FROM user_constraints WHERE table_name = 'DEPT';
 - SELECT * FROM user_cons_columns WHERE table_name = 'DEPT';

- -- 인덱스 정보
- SELECT * FROM user_indexes WHERE table_name = 'DEPT';
- SELECT * FROM user_ind_columns WHERE table_name = 'DEPT';

- INSERT INTO hr.dept(dept_id,dept_name) values (10,'총무부');
- INSERT INTO hr.dept(dept_id,dept_name) values (20,'분석팀');
- commit;

- SELECT * FROM hr.dept;

- desc hr.emp;

- ALTER TABLE hr.emp ADD dept_id NUMBER;

- INSERT INTO hr.emp(id,name,day,dept_id) values (1,'Liam',sysdate,10);
- INSERT INTO hr.emp(id,name,day,dept_id) values (2,'James',sysdate,30);
- commit;

- select e.*,d.*
- from hr.emp e, hr.dept d
- where e.dept_id = d.dept_id(+);

- DELETE FROM hr.emp;
- COMMIT;

- SELECT * FROM hr.emp;

- -- foreign key
 - -- 외래키 참조 무결성
 - -- 동일한 테이블이나 다른 테이블의 primary key, unique key 제약조건
 - -- 중복값 허용, NULL 값 허용
 - -- 종속되는 행 삭제 불가
 - -- 종속되는 테이블 삭제 불가

 - ALTER TABLE hr.emp ADD CONSTRAINT demp_dept_id_fk FOREIGN KEY(dept_id) REFERENCES hr.dept(dept_id);

 - -- 제약조건 정보
 - SELECT * FROM all_constraints WHERE table_name IN ('DEPT','EMP');
 - SELECT * FROM all_cons_columns WHERE table_name IN('DEPT','EMP');

 - INSERT INTO hr.emp(id,name,day,dept_id) values (1,'Liam',sysdate,10);
 - INSERT INTO hr.emp(id,name,day,dept_id) values (2,'James',sysdate,30); -- 오류발생 ,메인키에 없는 데이터 입력 + 외래키 조건 위반

 - SELECT * FROM hr.emp;
 - ROLLBACK;

- SELECT * FROM hr.dept;
- DELETE FROM hr.dept WHERE dept_id = 10; -- 현재 외래키 참조로 인해 삭제 불가
- DELETE FROM hr.dept WHERE dept_id = 20; -- 참조 중이어도 자식 데이터가 없으면 삭제 가능
- ROLLBACK;
- DROP TABLE hr.dept PURGE; -- 참조하고 있는 외래키 제약조건이 있어 불가능

DROP TABLE hr.dept CASCADE CONSTRAINT PURGE;

- CREATE TABLE hr.emp(id number, name varchar2(30), day date, dept_id number) TABLESPACE users;
- ALTER TABLE hr.emp ADD CONSTRAINT emp_id_pk PRIMARY KEY(id);
- CREATE TABLE hr.dept(dept_id number, dept_name varchar2(30)) TABLESPACE users;
- ALTER TABLE hr.dept ADD CONSTRAINT dept_pk PRIMARY KEY(dept_id);
- INSERT INTO hr.dept(dept_id,dept_name) values(10,'총무부');
- INSERT INTO hr.dept(dept_id,dept_name) values(20,'분석팀');
- INSERT INTO hr.emp(id,name,day,dept_id) values(1,'Liam',sysdate,10);
- commit;
- -- 제약조건 정보
- SELECT * FROM all_constraints WHERE table_name IN ('DEPT','EMP');
- SELECT * FROM all_cons_columns WHERE table_name IN('DEPT','EMP');
- ALTER TABLE hr.dept DROP CONSTRAINT dept_pk;
- ALTER TABLE hr.dept DROP PRIMARY KEY;
- ALTER TABLE hr.dept DROP CONSTRAINT dept_pk; -- 참고하고 있는 외래키가 있으면 오류발생
- -- 해결방법
- ALTER TABLE hr.emp DROP CONSTRAINT emp_dept_id_fk;
- -- &
- ALTER TABLE hr.dept DROP CONSTRAINT dept_pl;
- -- &
- ALTER TABLE hr.dept DROP CONSTRAINT dept_pl CASCADE;
- SELECT * FROM all_constraints WHERE table_name IN ('DEPT','EMP');
- -- UNIQUE 제약조건
 - -- 유일한 값만 체크

- -- 중복성 값 허용 x, NULL 허용
- -- 자동으로 UNIQUE INDEX 생성

- SELECT * FROM hr.dept;
- ALTER TABLE hr.dept ADD CONSTRAINT dept_name_uk
UNIQUE(dept_name);

- -- 제약조건 정보
- SELECT * FROM all_constraints WHERE table_name IN ('DEPT','EMP');
- SELECT * FROM all_cons_columns WHERE table_name IN ('DEPT','EMP');

- -- 인덱스 정보
- SELECT * FROM all_indexes WHERE table_name IN ('DEPT','EMP');
- SELECT * FROM all_ind_columns WHERE table_name IN ('DEPT','EMP');

- INSERT INTO hr.dept(dept_id, dept_name) values (30,'총무부'); -- UNIQUE
제약조건 위반 오류 중복성 발생
- INSERT INTO hr.dept(dept_id, dept_name) values (30,null); -- null은 허용
- ROLLBACK;

- -- UNIQUE 제약 조건 삭제
- ALTER TABLE hr.dept DROP CONSTRAINT dept_name_uk;
- ALTER TABLE hr.dept DROP UNIQUE(dept_name);

- ALTER TABLE hr.emp ADD sal number;

- -- CHECK 제약조건
 - -- 조건에 값이 TRUE 인 경우 INSERT, UPDATE 할 수 있도록 하는 제약조건
 - -- NULL 허용

 - ALTER TABLE hr.emp ADD CONSTRAINT emp_sal_ck CHECK(sal >= 1000
AND sal <= 2000);
 - ALTER TABLE hr.emp ADD CONSTRAINT emp_sal_ck CHECK (sal
BETWEEN 1000 AND 2000);

 - INSERT INTO hr.emp(id, name , day, dept_id, sal) values
(2,'Scott',sysdate,20,3000); -- CHECK 제약조건 위반 '3000'
 - INSERT INTO hr.emp(id, name , day, dept_id, sal) values
(2,'Scott',sysdate,20,1800);
 - ROLLBACK;

 - UPDATE hr.emp -- 제약조건 위반된 데이터가 수정하게 되면 오류발생
 - SET sal = 10000
 - WHERE id = 1;

 - DELETE FROM hr.emp WHERE id = 1;

 - -- 삭제

- ALTER TABLE hr.emp DROP CONSTRAINT emp_sal_ck;
- -- NOT NULL 제약조건
- -- NULL값 불허
- -- 중복성 허용
- INSERT INTO hr.emp (id,name,day,dept_id,sal)
values(2,null,sysdate,20,1000);
- ROLLBACK;
- ALTER TABLE hr.emp ADD CONSTRAINT emp_name_nn NOT
NULL(name); -- ADD CONSTRAINT 오류 발생 > modify
- ALTER TABLE hr.emp MODIFY name CONSTRAINT emp_name_nn NOT
NULL;
- -- 삭제
- ALTER TABLE hr.emp DROP CONSTRAINT emp_name_nn;
- ALTER TABLE hr.emp MODIFY name NULL;
- DROP TABLE hr.emp CASCADE CONSTRAINT PURGE;
- CREATE TABLE hr.emp(
○ id number, --열 레벨
○ name varchar2(30) CONSTRAINT emp_name_nn NOT NULL, -- 열 레벨
NOT NULL은 무조건 열레벨로 해야 된다.
○ sal number,
○ dept_id number CONSTRAINT emp_dept_id_fk REFERENCES
hr.dept(dept_id), -- 열 레벨
○ CONSTRAINT emp_id_pk PRIMARY KEY(id),
○ CONSTRAINT emp_name_uk UNIQUE(name), -- 테이블 레벨
○ CONSTRAINT emp_sal_ck CHECK(sal BETWEEN 1000 AND 2000))
○ TABLESPACE users;
- SELECT * FROM all_constraints WHERE table_name IN ('DEPT','EMP');
- CREATE TABLE hr.emp(
○ id number,
○ name varchar2(30)
○ CONSTRAINT emp_name_nn NOT NULL,
○ sal number,
○ dept_id number,
○ CONSTRAINT emp_id_pk PRIMARY KEY(id),
○ CONSTRAINT emp_name_uk UNIQUE(name),
○ CONSTRAINT emp_sal_ck CHECK(sal BETWEEN 1000 AND 2000),
○ CONSTRAINT emp_dept_id_fk FOREIGN KEY (dept_id)

- REFERENCES hr.dept(dept_id)
-)
- TABLESPACE users;
- INSERT INTO hr.emp(id,name,dept_id,sal) values (1,'James',20,1000);
- commit;
- -- 테이블 이름 수정
- -- 소유자만 수행
- RENAME hr.emp TO hr.emp_new; -- 소유자 이름 표현 x
- RENAME emp TO emp_new;
- SELECT * FROM hr.emp;
- ALTER TABLE hr.emp_new RENAME TO emp;
- -- 컬럼이름 수정
- ALTER TABLE hr.emp RENAME COLUMN id TO emp_id;
- desc hr.emp;
- -- 제약조건 이름 수정
- ALTER TABLE hr.emp RENAME CONSTRAINT emp_id_pk TO emp_pk;
- -- 인덱스 이름 수정
- ALTER INDEX emp_id_pk RENAME TO emp_idx;
- GRANT SELECT ON hr.employees TO insa;
- REVOKE SELECT ON hr.employees FROM insal
- SELECT * FROM user_tab_privs;
- CREATE TABLE hr.copy_emp
- AS
- SELECT employee_id, last_name, job_id, department_id
- FROM hr.employees;
- GRANT SELECT ON hr.copy_emp TO insa;
- SELECT * FROM user_tab_privs;
- SELECT * FROM hr.employees;
- SELECT * FROM hr.copy_emp;

- DROP TABLE hr.copy_emp PURGE;
- CREATE OR REPLACE VIEW hr.copy_emp
- AS
- SELECT employee_id, last_name, job_id, department_id
- FROM hr.employees;
- -- VIEW
 - -- 하나 이상의 테이블이 있는 데이터를 논리적으로 처리하는 객체
 - -- 단지 SELECT 문만 가지고 있는 객체
 - SELECT * FROM all_objects WHERE object_name IN ('EMPLOYEES','COPY_EMP');
 - SELECT * FROM all_views WHERE view_name = 'COPY_EMP';
 - SELECT employee_id, last_name, job_id, department_id
 - FROM hr.employees;
 - SELECT * FROM all_objects;
 - SELECT * FROM all_views WHERE view_name = 'COPY_EMP';
 - SELECT * FROM all_views WHERE view_name = 'USER_OBJECTS';
 - -- VIEW 삭제
 - DROP VIEW hr.copy_emp;
 - -- VIEW 생성
 - CREATE VIEW
 - SELECT * FROM session_privs;
 - CREATE OR REPLACE VIEW hr.copy_emp
 - AS
 - SELECT employee_id, last_name||' '|| job_id name, department_id --
표현식의 일부가 되어 있다면 별칭을 써줘야 된다.
 - FROM hr.employees;
 - -- VIEW 는 수정이 안되어 삭제 후 다시 생성해야 된다.
 - -- OR REPLACE - 기존 뷰가 있으면 삭제한 후 다시 생성
 - DROP VIEW hr.copy_emp;
 - -- DROP 후에는 그 뷰에 대해서 다시 권한부여를 해줘야 된다.
 - CREATE OR REPLACE VIEW hr.copy_emp
 - (id, name, job, dept_id)
 - AS

- SELECT employee_id,last_name,job_id,department_id
- FROM hr.employees;

desc hr.copy_emp;

- SELECT * FROM hr.copy_emp;
- GRANT SELECT ON hr.copy_emp TO insa;
- SELECT * FROM all_tab_privs;
- CREATE OR REPLACE VIEW hr.copy_emp
- AS
- SELECT e.employee_id,e.last_name,e.first_name,e.job_id,e.department_id,
d.department_name
- FROM hr.employees e ,hr.departments d
- WHERE e.department_id = d.department_id;
-